Divide Array into Groups of 5 Consecutive Numbers

Company: Google
Round: Interview Round
Year: 2026
Difficulty: Medium
Topic: Arrays, HashMap, Sorting, Frequency Counting
Source: https://www.desiqna.in/19213/google-interview-experiences-sde-2026-set-74

Problem Description

Given an array of N integers and an integer 5, determine whether the array can be divided into groups of exactly 5 elements such that every group contains 5 consecutive increasing integers.

Each element of the array must be used exactly once. Duplicate values are allowed, but they must be handled correctly.

Return true if the array can be completely divided into such groups; otherwise, return false.

Input Format

The first line contains N, the number of elements in the array.

The second line contains N space-separated integers representing the elements of the array.

Output Format

Print true if the array can be divided into groups of exactly 5 consecutive increasing integers, using every element exactly once.

Otherwise, print false.

Sample Input

10
1 2 3 4 5 6 7 8 9 10

Sample Output

true

Explanation

We need to divide the array into groups of exactly 5 consecutive numbers.
First, we count the frequency of each number so that duplicates are handled correctly.
We start with the smallest unused number and check whether the next 4 consecutive numbers are available.
If any required number is missing, we cannot form a valid group, so we return false.
We decrease the frequency of all five numbers after forming each group and continue until all elements are used.
If every element is successfully grouped, we return true.

Approach to Solve This Problem:

1) Check the array size:
Since every group must contain exactly 5 elements, N must be divisible by 5. If not, return false.

2) Count the frequency of each number:
Use a HashMap to store how many times each number appears. This is important because the array may contain duplicates.

3) Find the smallest unused number:
Start with the smallest number whose frequency is still greater than 0. This number must be the starting point of a group because there is no smaller unused number available.

4) Try to form a group of 5:
For a starting number x, check whether x, x+1, x+2, x+3, x+4 are all available. If any number is missing, return false.

5) Use those elements:
Decrease the frequency of all five numbers because they have now been used. Then find the next smallest unused number and repeat the process.

6) Return the result:
If all elements are successfully used to form groups of 5 consecutive numbers, return true; otherwise, return false.

Java Solution:

Main.java > Main

```java
import java.util.*;

public class Main {

    public static boolean canDivide(int[] arr) {

        int n = arr.length;

        // Number of elements must be divisible by 5
        if (n % 5 != 0) {
            return false;
        }

        // Count frequency of each number
        HashMap<Integer, Integer> freq = new HashMap<>();

        for (int num : arr) {
            freq.put(num, freq.getOrDefault(num, defaultValue: 0) + 1);
        }

        // Sort the array to process numbers from smallest to largest
        Arrays.sort(arr);

        // Try to form groups of 5 consecutive numbers
        for (int num : arr) {

            // If this number is already completely used, skip it
            if (freq.get(num) == 0) {
                continue;
            }

            // Try to form: num, num+1, num+2, num+3, num+4
            for (int i = 0; i < 5; i++) {

                int current = num + i;

                // Required number is not available
                if (freq.getOrDefault(current, defaultValue: 0) == 0) {
                    return false;
                }

                // Use one occurrence of this number
                freq.put(current, freq.get(current) - 1);
            }
        }

        return true;
    }
```

```
49
        Run | Debug
50      public static void main(String[] args) {
51
52          Scanner sc = new Scanner(System.in);
53
54          int n = sc.nextInt();
55
56          int[] arr = new int[n];
57
58          for (int i = 0; i < n; i++) {
59              arr[i] = sc.nextInt();
60          }
61
62          System.out.println(canDivide(arr));
63
64          sc.close();
65      }
66  }
```

Solution:

1. Check if groups of 5 are possible:

- Every group must contain exactly 5 elements. Therefore, the total number of elements must be divisible by 5.
- If n % 5 != 0, we cannot divide the array completely, so return false.

2. Count the frequency of each number:

- Use a HashMap to store how many times each number appears in the array.
- This helps us handle duplicate numbers correctly. For example, if 3 appears twice, the map stores 3 → 2.

3. Form groups using the smallest unused number:

- Sort the array so that we can process numbers from smallest to largest.
- Take the smallest number that is still unused and consider it as the start of a group.
- Check whether the next four consecutive numbers are available. For example, if the starting number is 3, we need 3, 4, 5, 6, 7.
- If any required number is missing, return false. Otherwise, decrease their frequencies because those elements have been used.

4. Final result:

- Continue forming groups until all elements are used.
- If every element can be placed into a group of 5 consecutive numbers, return true.
- Otherwise, return false. The solution correctly handles duplicates and ensures that every element is used exactly once.

Time Complexity:- O(N log N)